Practical File

On

DATABASE MANAGEMENT SYSTEMS

Submitted in partial fulfilment of the requirements for
the award of the degree

Bachelor of Technology

in

Department of Computer Science and Engineering

Submitted to:

Dr. Poonam Sangwan

Assistant Professor

CSE/SOET

Submitted by:

Harshit Khemani

241302081

B.Tech CSE (AI/ML)

Section - C

5th Semester

INDEX

S. No.	Name of Experiment	Date	Sign.
1			
2			
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			

Ex – 1

Employee Directory

1. Aim

To design a relational schema for an employee directory with departments and employees, enforce primary-key and foreign-key constraints, create a reporting view, and verify the design with sample queries in SQLite.

2. Theory

2.1 Tables and keys

A `departments` table stores each organisational unit. An `employees` table stores people; `department_id` is a foreign key to `departments`, and `manager_id` is a self-referencing foreign key for the reporting hierarchy.

2.2 Views

A view (`employee_directory`) joins employees to their department and manager names so reporting queries do not repeat join logic.

3. Schema

Table	Key columns	Role
departments	department_id PK	Organisational units
employees	employee_id PK; department_id FK	Staff records
employees	manager_id FK → employees	Reporting chain
employee_directory	VIEW	Flattened directory for queries

4. Procedure

1. Create `departments` and `employees` with constraints and indexes.
2. Create the `employee_directory` view.
3. Insert six departments and twenty-six employees with a three-tier org chart.
4. Run aggregate queries on the view (counts by department).

5. Source Code

File: 19-08-2026/schema.sql

```
-- ============================================================
-- Employee Directory — schema
-- SQLite dialect. Runs as-is in the in-browser SQL Compiler
-- (index.html), and in the sqlite3 CLI / DB Browser for SQLite.
-- ============================================================

PRAGMA foreign_keys = ON;

DROP VIEW IF EXISTS employee_directory;
DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;

-- ------------------------------------------------------------
-- departments
-- ------------------------------------------------------------
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name          TEXT NOT NULL UNIQUE,
```

```
    description   TEXT,
    color_hex     TEXT NOT NULL DEFAULT '#1CB0F6'
);

-- ------------------------------------------------------------
-- employees
-- ------------------------------------------------------------
CREATE TABLE employees (
    employee_id       INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name        TEXT NOT NULL,
    last_name         TEXT NOT NULL,
    email             TEXT NOT NULL UNIQUE,
    job_title         TEXT NOT NULL,
    department_id     INTEGER NOT NULL REFERENCES departments (department_id) ON DELETE RESTRICT,
    manager_id        INTEGER REFERENCES employees (employee_id) ON DELETE SET NULL,
    salary            NUMERIC(10,2) NOT NULL CHECK (salary > 0),
    hire_date         TEXT NOT NULL,
    employment_status TEXT NOT NULL DEFAULT 'active'
                        CHECK (employment_status IN ('active', 'remote', 'on_leave')),
    avatar_seed       TEXT NOT NULL
);

CREATE INDEX idx_employees_department ON employees (department_id);
CREATE INDEX idx_employees_manager    ON employees (manager_id);

-- ------------------------------------------------------------
-- employee_directory — flattened view joining department + manager.
-- Convenient for reporting queries and reused by the frontend.
-- ------------------------------------------------------------
CREATE VIEW employee_directory AS
SELECT
    e.employee_id,
    e.first_name || ' ' || e.last_name AS full_name,
    e.email,
    e.job_title,
    d.name                             AS department_name,
    d.color_hex                        AS department_color,
    m.first_name || ' ' || m.last_name AS manager_name,
    e.salary,
    e.hire_date,
    e.employment_status,
    e.avatar_seed
FROM employees e
JOIN departments d ON d.department_id = e.department_id
LEFT JOIN employees m ON m.employee_id = e.manager_id;
```

6. Output

```
SQL> SELECT COUNT(*) AS dept_count FROM departments;
dept_count
6

SQL> SELECT COUNT(*) AS emp_count FROM employees;
emp_count
26

SQL> SELECT department_name, COUNT(*) AS headcount
     FROM employee_directory
     GROUP BY department_name
     ORDER BY headcount DESC;
department_name  headcount
Engineering      8
Design           5
Marketing        4
Sales            4
People Ops       3
Finance          2

SQL> SELECT full_name, department_name, manager_name
     FROM employee_directory
     LIMIT 5;
full_name           department_name  manager_name
Sam Okafor          Engineering      NULL
Priya Nair          Engineering      Sam Okafor
Elena Petrova       Design           Sam Okafor
Grace Adeyemi       Marketing        Sam Okafor
Isabella Rossi      Sales            Sam Okafor
```

7. Results

Six departments and twenty-six employees load successfully. Foreign keys prevent orphan `department_id` values. The view returns each employee with department and manager names for browsing and SQL practice.

8. Conclusion

Normalised tables plus a view provide a maintainable employee directory. Primary and foreign keys preserve referential integrity; the view simplifies read-only reporting queries used in the web compiler.

Ex – 2

CREATE & ALTER TABLE

1. Aim

To create an `employees` table with attributes `empID`, `empName`, `department` and `salary`, insert sample records, and then rename the column `department` to `faculty` using `ALTER TABLE`.

2. Theory

2.1 CREATE TABLE

A relational table is defined by a name, a list of attributes, and constraints that protect the data. `PRIMARY KEY` makes `empID` unique and not null. `NOT NULL` rejects missing names and faculty values. `CHECK (salary > 0)` rejects non-positive salaries. `NUMERIC(10, 2)` stores pay as a decimal with two fractional digits.

2.2 ALTER TABLE RENAME COLUMN

In SQLite and MySQL 8+, `ALTER TABLE ... RENAME COLUMN` changes a column's name without rewriting the stored values. After `RENAME COLUMN department TO faculty`, every row still holds the same school name; only the heading of that column becomes `faculty`. This is a schema change, not an `UPDATE`. Microsoft SQL Server does not accept that syntax; the equivalent is `EXEC sp_rename 'employees.department', 'faculty', 'COLUMN'` (see `employees.sqlserver.sql`).

3. Schema

The table is created with four attributes. After the rename, the third attribute is `faculty` instead of `department`.

Attribute	Type	Constraint	Role
empID	INTEGER	PRIMARY KEY	Employee identifier
empName	TEXT	NOT NULL	Employee name
department → faculty	TEXT	NOT NULL	School / faculty name
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary

Sample faculty values include **School of Engineering and Technology** (the faculty named on this report's cover). The cover also lists **Department of CSE** as the student's home department; that heading is letterhead, not a fifth column in the table.

4. Procedure

1. Drop `employees` if it already exists, so the script can be re-run.
2. Create the table with `empID`, `empName`, `department` and `salary`.
3. Insert six sample employees.
4. Display all rows and the column list with `PRAGMA table_info`.
5. Rename `department` to `faculty`.
6. Display all rows and the column list again to confirm that only the name changed.

5. Source Code

```
-- ============================================================
-- DBMS Lab · 26-08-2026
-- Employees table: CREATE, seed, then RENAME department → faculty
```

```
-- SQLite  (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employees.sqlserver.sql instead.
-- ============================================================

DROP TABLE IF EXISTS employees;

-- ------------------------------------------------------------
-- 1. Create the employees table
-- ------------------------------------------------------------
CREATE TABLE employees (
    empID      INTEGER PRIMARY KEY,
    empName    TEXT NOT NULL,
    department TEXT NOT NULL,
    salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);

-- ------------------------------------------------------------
-- 2. Insert sample rows (department holds the faculty/school name)
-- ------------------------------------------------------------
INSERT INTO employees (empID, empName, department, salary) VALUES
    (1, 'Ananya Sharma',  'School of Engineering and Technology', 92000.00),
    (2, 'Rohan Mehta',    'School of Management',                 78000.00),
    (3, 'Priya Nair',     'School of Law',                        85000.00),
    (4, 'Vikram Singh',   'School of Agricultural Sciences',      76000.00),
    (5, 'Fatima Khan',    'School of Engineering and Technology', 88000.00),
    (6, 'Arjun Patel',    'School of Medical Sciences',           95000.00);

-- ------------------------------------------------------------
-- 3. Verify the table BEFORE the rename
-- ------------------------------------------------------------
SELECT * FROM employees;
PRAGMA table_info(employees);

-- ------------------------------------------------------------
-- 4. Rename the column department → faculty
--    Row values are unchanged; only the column name changes.
-- ------------------------------------------------------------
ALTER TABLE employees RENAME COLUMN department TO faculty;

-- ------------------------------------------------------------
-- 5. Verify the table AFTER the rename
-- ------------------------------------------------------------
SELECT * FROM employees;
PRAGMA table_info(employees);
```

6. Output

The script was executed in SQLite. `PRAGMA table_info` columns are `cid` (column index), `name`, `type`, `notnull`, `dflt_value` and `pk`.

```
SQL> DROP TABLE IF EXISTS employees;
-- OK

SQL> CREATE TABLE employees (
         empID      INTEGER PRIMARY KEY,
         empName    TEXT NOT NULL,
         department TEXT NOT NULL,
         salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
     );
-- OK

SQL> INSERT INTO employees (empID, empName, department, salary) VALUES
         (1, 'Ananya Sharma',  'School of Engineering and Technology', 92000.00),
         (2, 'Rohan Mehta',    'School of Management',                 78000.00),
         (3, 'Priya Nair',     'School of Law',                        85000.00),
         (4, 'Vikram Singh',   'School of Agricultural Sciences',      76000.00),
         (5, 'Fatima Khan',    'School of Engineering and Technology', 88000.00),
         (6, 'Arjun Patel',    'School of Medical Sciences',           95000.00);
-- 6 row(s) inserted

SQL> SELECT * FROM employees;
empID  empName        department                          salary
-----  -------------  ----------------------------------  ------
1      Ananya Sharma  School of Engineering and Technology  92000
2      Rohan Mehta    School of Management                 78000
3      Priya Nair     School of Law                        85000
4      Vikram Singh   School of Agricultural Sciences      76000
5      Fatima Khan    School of Engineering and Technology  88000
6      Arjun Patel    School of Medical Sciences           95000

SQL> PRAGMA table_info(employees);
cid  name        type            notnull  dflt_value  pk
---  ----------  --------------  -------  ----------  --
0    empID       INTEGER         0                    1
```

```
1    empName     TEXT            1                   0
2    department  TEXT            1                   0
3    salary      NUMERIC(10, 2)  1                   0

SQL> ALTER TABLE employees RENAME COLUMN department TO faculty;
-- OK

SQL> SELECT * FROM employees;
empID  empName        faculty                              salary
-----  -------------  -----------------------------------  ------
1      Ananya Sharma  School of Engineering and Technology  92000
2      Rohan Mehta    School of Management                 78000
3      Priya Nair     School of Law                        85000
4      Vikram Singh   School of Agricultural Sciences      76000
5      Fatima Khan    School of Engineering and Technology  88000
6      Arjun Patel    School of Medical Sciences           95000

SQL> PRAGMA table_info(employees);
cid  name     type            notnull  dflt_value  pk
---  -------  --------------  -------  ----------  --
0    empID    INTEGER         0                    1
1    empName  TEXT            1                    0
2    faculty  TEXT            1                    0
3    salary   NUMERIC(10, 2)  1                    0
```

7. Results

Stage	Third column	Row count	Values changed?
Before ALTER	department	6	No — original schema
After ALTER	faculty	6	No — names only

All six employees remain. School names such as School of Engineering and Technology are unchanged. The schema listing is the proof: `cid = 2` is named `department` before the statement and `faculty` afterwards.

8. Conclusion

An employees relation can be created with a primary key, not-null attributes and a salary check, then adjusted with `ALTER TABLE ... RENAME COLUMN`. Renaming `department` to `faculty` updates the schema only. The stored values do not change, which is the correct way to relabel an attribute without rewriting the table.

Ex – 3

SQL Joins

1. Aim

To combine rows from related tables using `CROSS JOIN`, `NATURAL JOIN`, `INNER JOIN`, `LEFT OUTER JOIN`, `RIGHT OUTER JOIN`, and `SELF JOIN`; to compare how each join type treats unmatched rows; and to understand when to use each one.

2. Theory

2.1 CROSS JOIN

A `CROSS JOIN` returns the Cartesian product of two tables: every row from the first table is paired with every row from the second. There is no join condition. With 6 departments and 7 employees the result has 6 × 7 = **42 rows**. Cross joins are rare in everyday queries but useful for generating combinations (e.g. all size–colour pairs) or as a building block for other joins.

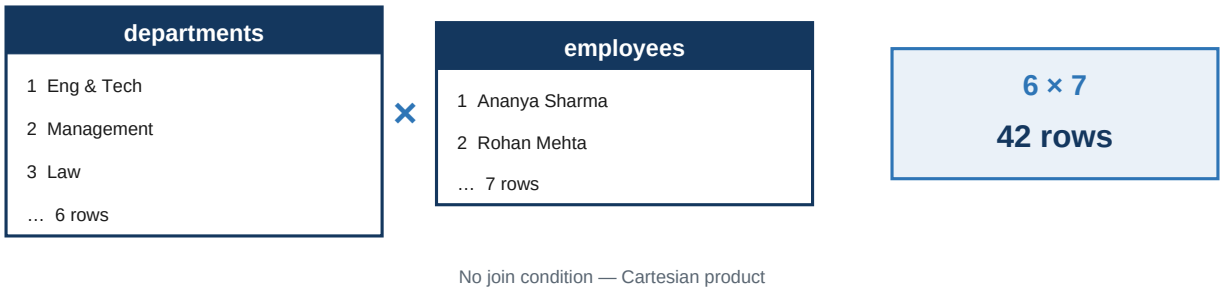

Figure 1 — CROSS JOIN pairs every department with every employee (42 combinations).

2.2 NATURAL JOIN

A `NATURAL JOIN` automatically matches rows on every column that appears in both tables with the same name. Here, both tables share `deptID`, so the condition is implicit. Only rows with a matching `deptID` in both tables are returned — the same six rows as `INNER JOIN` when `deptID` is the sole common column. (SQL Server does not support `NATURAL JOIN`; use an explicit `INNER JOIN` instead.)

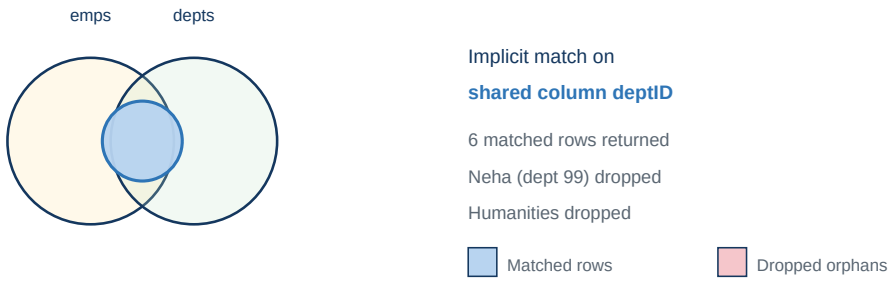

Figure 2 — NATURAL JOIN keeps only rows where `deptID` matches in both tables.

2.3 INNER JOIN

An `INNER JOIN` returns only rows where the explicit join condition is satisfied. Unmatched rows from either table are discarded. It is the most common join: use it when you only want records that have a partner on both sides.

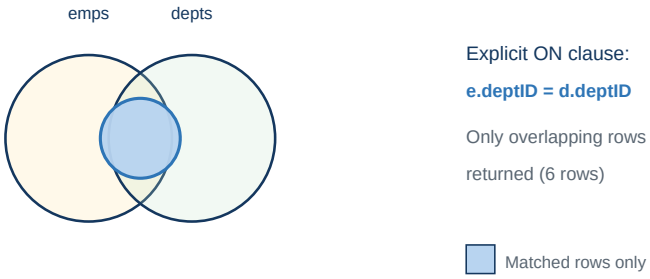

Figure 3 — INNER JOIN returns only the overlapping region (6 matched rows).

2.4 LEFT OUTER JOIN

A `LEFT OUTER JOIN` keeps every row from the left table. If no match exists in the right table, the right-hand columns are filled with `NULL`. Employee `Neha Gupta` (deptID 99) has no matching department and therefore appears with a null department name.

employees (left table)

Ananya	dept=1
Rohan	dept=2
Priya	dept=3
Neha	dept=99 ← no match

LEFT JOIN result

Ananya	Eng & Tech
Rohan	Management
Priya	Law
Neha	NULL

All left rows kept — unmatched right columns print as NULL

Figure 4 — LEFT OUTER JOIN preserves Neha (deptID 99); `deptName` is NULL.

2.5 RIGHT OUTER JOIN

A `RIGHT OUTER JOIN` keeps every row from the right table. Unmatched left-hand columns become `NULL`. `School of Humanities` (deptID 6) has no employees and therefore appears with null employee fields. A `RIGHT JOIN` can always be rewritten as a `LEFT JOIN` by swapping the table order.

departments (right table)

1	Eng & Tech
2	Management
3	Law
6	Humanities ← no employees

RIGHT JOIN result

Ananya	Eng & Tech
Rohan	Management
Priya	Law
NULL	Humanities

All right rows kept — unmatched left columns print as NULL

Figure 5 — RIGHT OUTER JOIN preserves Humanities (deptID 6); employee columns are NULL.

2.6 SELF JOIN

A `SELF JOIN` joins a table to itself using two different aliases. It is not a separate SQL keyword — it is an `INNER` or `LEFT JOIN` where both sides are the same table. Here, `employees e` is joined to `employees m` on `e.managerID = m.empID` to list each employee alongside their manager's name.

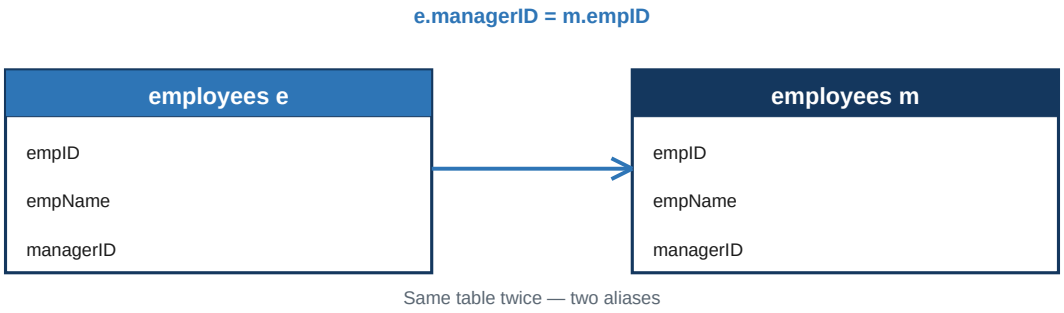

Figure 6 — SELF JOIN: the same table appears twice with different aliases.

2.7 Differences between all join types

Join type	Condition	Unmatched rows	Typical use
CROSS JOIN	None (Cartesian product)	N/A — all combinations	Generate all pairs
NATURAL JOIN	Implicit (shared names)	Dropped from both sides	Quick join on a shared key
INNER JOIN	Explicit ON clause	Dropped from both sides	Only matching records
LEFT OUTER JOIN	Explicit ON clause	Left kept; right NULL	Keep all from the left table
RIGHT OUTER JOIN	Explicit ON clause	Right kept; left NULL	Keep all from the right table
SELF JOIN	Explicit ON (same table)	Depends on INNER / LEFT	Hierarchies, comparisons

Key distinctions: `CROSS JOIN` multiplies rows with no filter. `INNER` and `NATURAL` return only matches. `LEFT` and `RIGHT OUTER` preserve one side's orphans. `SELF JOIN` is a technique (same table, two aliases), not a separate result category — it uses inner or outer join semantics on a single table.

3. Schema

Two tables are linked by `deptID`. `employees` also references itself through `managerID` for the self join (see Figure 6).

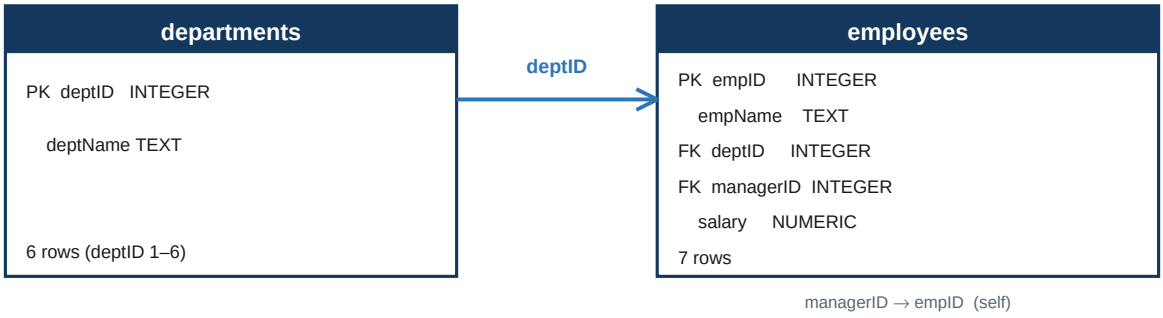

Figure 7 — Schema: `employees.deptID` references `departments.deptID`; `managerID` is a self-reference.

Table	Attribute	Type	Role
departments	deptID	INTEGER / INT	Primary key
departments	deptName	TEXT / VARCHAR	School name
employees	empID	INTEGER / INT	Primary key
employees	empName	TEXT / VARCHAR	Employee name
employees	deptID	INTEGER / INT	FK → departments
employees	managerID	INTEGER / INT	FK → employees (self)
employees	salary	NUMERIC(10,2)	Monthly salary

4. Procedure

1. Create `departments` and `employees` (with `managerID` for self join).
2. Insert six departments and seven employees, including deliberate mismatches for outer joins.
3. Display both base tables.
4. Run `CROSS JOIN` — expect 42 rows (count shown; sample limited to 8).
5. Run `NATURAL JOIN` and `INNER JOIN` — expect six matched rows.
6. Run `LEFT OUTER JOIN` — expect seven rows (one employee without a department).
7. Run `RIGHT OUTER JOIN` — expect seven rows (one department without employees).
8. Run `SELF JOIN` — expect seven rows listing each employee and their manager.

5. Source Code

SQLite script used in the lab compiler (`02-09-2026/joins.sql`). For the class SQL Server, run `joins.sqlserver.sql`.

```
-- ============================================================
-- DBMS Lab · 02-09-2026
-- SQL Joins: CROSS, NATURAL, INNER, OUTER (LEFT/RIGHT), SELF
-- SQLite  (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run joins.sqlserver.sql instead.
-- ============================================================

DROP TABLE IF EXISTS employees;
DROP TABLE IF EXISTS departments;

-- ------------------------------------------------------------
-- 1. Create related tables
--    • deptID links employees → departments
--    • managerID links employees → employees (for SELF JOIN)
-- ------------------------------------------------------------
CREATE TABLE departments (
    deptID   INTEGER PRIMARY KEY,
    deptName TEXT NOT NULL
);

CREATE TABLE employees (
    empID      INTEGER PRIMARY KEY,
    empName    TEXT NOT NULL,
    deptID     INTEGER NOT NULL,
    managerID  INTEGER,
    salary     NUMERIC(10, 2) NOT NULL CHECK (salary > 0)
);

-- ------------------------------------------------------------
-- 2. Insert sample rows
--    • deptID 6 (Humanities) has no employees  → RIGHT OUTER JOIN
--    • empID 7 points to deptID 99 (missing)   → LEFT OUTER JOIN
--    • managerID links form a simple reporting chain → SELF JOIN
-- ------------------------------------------------------------
INSERT INTO departments (deptID, deptName) VALUES
    (1, 'School of Engineering and Technology'),
    (2, 'School of Management'),
    (3, 'School of Law'),
    (4, 'School of Agricultural Sciences'),
    (5, 'School of Medical Sciences'),
    (6, 'School of Humanities');

INSERT INTO employees (empID, empName, deptID, managerID, salary) VALUES
    (1, 'Ananya Sharma', 1, NULL,  92000.00),
    (2, 'Rohan Mehta',   2, 1,     78000.00),
    (3, 'Priya Nair',    3, 1,     85000.00),
    (4, 'Vikram Singh',  4, 2,     76000.00),
    (5, 'Fatima Khan',   1, 1,     88000.00),
    (6, 'Arjun Patel',   5, 3,     95000.00),
    (7, 'Neha Gupta',   99, 2,     54000.00);

-- ------------------------------------------------------------
-- 3. Base tables (reference)
-- ------------------------------------------------------------
-- List all departments
SELECT * FROM departments;
-- List all employees
SELECT * FROM employees;

-- ------------------------------------------------------------
-- 4. CROSS JOIN
--    Cartesian product: every row of table A paired with every
--    row of table B. No join condition. Here: 6 × 7 = 42 rows.
-- ------------------------------------------------------------
-- Count cross join rows
SELECT COUNT(*) AS cross_join_row_count
FROM departments
CROSS JOIN employees;

-- Cross join sample pairs
SELECT d.deptName, e.empName
FROM departments AS d
CROSS JOIN employees AS e
ORDER BY d.deptID, e.empID
LIMIT 8;

-- ------------------------------------------------------------
-- 5. NATURAL JOIN
--    Matches on every column with the same name (here: deptID).
--    Only rows with a matching deptID in BOTH tables appear.
-- ------------------------------------------------------------
```

```sql
-- Natural join on deptID
SELECT empID, empName, deptName, salary
FROM employees
NATURAL JOIN departments;

-- ------------------------------------------------------------
-- 6. INNER JOIN
--    Explicit join condition; same result as NATURAL JOIN here
--    because deptID is the only common column.
-- ------------------------------------------------------------
-- Inner join employees and departments
SELECT e.empID, e.empName, d.deptName, e.salary
FROM employees AS e
INNER JOIN departments AS d ON e.deptID = d.deptID;

-- ------------------------------------------------------------
-- 7a. LEFT OUTER JOIN
--     All rows from the LEFT table (employees); unmatched dept
--     columns are NULL (empID 7 → deptID 99 has no department).
-- ------------------------------------------------------------
-- Left join — keep all employees
SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
FROM employees AS e
LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;

-- ------------------------------------------------------------
-- 7b. RIGHT OUTER JOIN
--     All rows from the RIGHT table (departments); unmatched
--     employee columns are NULL (deptID 6 has no employees).
--     Requires SQLite 3.39+; see joins.sqlserver.sql for T-SQL.
-- ------------------------------------------------------------
-- Right join — keep all departments
SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
FROM employees AS e
RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;

-- ------------------------------------------------------------
-- 8. SELF JOIN
--     A table joined to itself using different aliases.
--     Here: each employee matched to their manager (also in
--     employees). Ananya has no manager → manager name is NULL.
-- ------------------------------------------------------------
-- Self join — employee to manager
SELECT
    e.empName  AS employee,
    m.empName  AS manager
FROM employees AS e
LEFT JOIN employees AS m ON e.managerID = m.empID
ORDER BY e.empID;
```

6. Output

Each `SELECT` from the script, executed in SQLite. Unmatched columns print as `NULL` (highlighted). CREATE and INSERT output is omitted here; the statements are in section 5.

6.1 Base table — departments

```
SQL> SELECT * FROM departments;
```

deptID	deptName
1	School of Engineering and Technology
2	School of Management
3	School of Law
4	School of Agricultural Sciences
5	School of Medical Sciences
6	School of Humanities

6.2 Base table — employees

```
SQL> SELECT * FROM employees;
```

empID	empName	deptID	managerID	salary
1	Ananya Sharma	1	*NULL*	92000
2	Rohan Mehta	2	1	78000
3	Priya Nair	3	1	85000
4	Vikram Singh	4	2	76000
5	Fatima Khan	1	1	88000
6	Arjun Patel	5	3	95000
7	Neha Gupta	99	2	54000

6.3 CROSS JOIN — row count

```
SQL> SELECT COUNT(*) AS cross_join_row_count
     FROM departments
     CROSS JOIN employees;
```

cross_join_row_count
42

6.4 CROSS JOIN — first 8 pairs

```
SQL> SELECT d.deptName, e.empName
     FROM departments AS d
     CROSS JOIN employees AS e
     ORDER BY d.deptID, e.empID
     LIMIT 8;
```

deptName	empName
School of Engineering and Technology	Ananya Sharma
School of Engineering and Technology	Rohan Mehta
School of Engineering and Technology	Priya Nair
School of Engineering and Technology	Vikram Singh
School of Engineering and Technology	Fatima Khan
School of Engineering and Technology	Arjun Patel
School of Engineering and Technology	Neha Gupta
School of Management	Ananya Sharma

6.5 NATURAL JOIN

```
SQL> SELECT empID, empName, deptName, salary
     FROM employees
     NATURAL JOIN departments;
```

empID	empName	deptName	salary
1	Ananya Sharma	School of Engineering and Technology	92000
2	Rohan Mehta	School of Management	78000
3	Priya Nair	School of Law	85000
4	Vikram Singh	School of Agricultural Sciences	76000
5	Fatima Khan	School of Engineering and Technology	88000
6	Arjun Patel	School of Medical Sciences	95000

6.6 INNER JOIN

```
SQL> SELECT e.empID, e.empName, d.deptName, e.salary
     FROM employees AS e
     INNER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptName	salary
1	Ananya Sharma	School of Engineering and Technology	92000
2	Rohan Mehta	School of Management	78000
3	Priya Nair	School of Law	85000
4	Vikram Singh	School of Agricultural Sciences	76000
5	Fatima Khan	School of Engineering and Technology	88000
6	Arjun Patel	School of Medical Sciences	95000

6.7 LEFT OUTER JOIN

```
SQL> SELECT e.empID, e.empName, e.deptID, d.deptName, e.salary
     FROM employees AS e
     LEFT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptID	deptName	salary
1	Ananya Sharma	1	School of Engineering and Technology	92000
2	Rohan Mehta	2	School of Management	78000
3	Priya Nair	3	School of Law	85000
4	Vikram Singh	4	School of Agricultural Sciences	76000
5	Fatima Khan	1	School of Engineering and Technology	88000
6	Arjun Patel	5	School of Medical Sciences	95000
7	Neha Gupta	99	*NULL*	54000

6.8 RIGHT OUTER JOIN

```
SQL> SELECT e.empID, e.empName, d.deptID, d.deptName, e.salary
     FROM employees AS e
     RIGHT OUTER JOIN departments AS d ON e.deptID = d.deptID;
```

empID	empName	deptID	deptName	salary
1	Ananya Sharma	1	School of Engineering and Technology	92000
2	Rohan Mehta	2	School of Management	78000
3	Priya Nair	3	School of Law	85000
4	Vikram Singh	4	School of Agricultural Sciences	76000
5	Fatima Khan	1	School of Engineering and Technology	88000
6	Arjun Patel	5	School of Medical Sciences	95000
NULL	*NULL*	6	School of Humanities	*NULL*

6.9 SELF JOIN — employee and manager

```
SQL> SELECT
         e.empName  AS employee,
         m.empName  AS manager
     FROM employees AS e
     LEFT JOIN employees AS m ON e.managerID = m.empID
     ORDER BY e.empID;
```

employee	manager
Ananya Sharma	*NULL*
Rohan Mehta	Ananya Sharma
Priya Nair	Ananya Sharma
Vikram Singh	Rohan Mehta
Fatima Khan	Ananya Sharma
Arjun Patel	Priya Nair
Neha Gupta	Rohan Mehta

7. Results

Row counts returned by each join on this dataset:

Join type	Rows	Unmatched handling
CROSS JOIN	42	No condition — every dept × every employee
NATURAL JOIN	6	Drops non-matching rows from both sides
INNER JOIN	6	Same six rows as NATURAL JOIN here
LEFT OUTER JOIN	7	Keeps Neha Gupta (deptID 99); deptName is NULL
RIGHT OUTER JOIN	7	Keeps Humanities (deptID 6); employee cols are NULL
SELF JOIN	7	All employees; Ananya has NULL manager (top of chain)

`CROSS JOIN` produces the largest result because it ignores relationships entirely. `INNER JOIN` and `NATURAL JOIN` return only the six employees whose `deptID` exists in `departments`. Outer joins add the orphan employee (left) or orphan department (right). `SELF JOIN` does not change row count here because a `LEFT JOIN` keeps every employee and simply leaves the manager column null for Ananya Sharma, who has no manager.

8. Conclusion

SQL joins answer different questions about related data. `CROSS JOIN` asks “every combination?” `INNER JOIN` asks “only matches?” Outer joins ask “keep everyone from this side even without a match?” `SELF JOIN` asks “how do rows in this table relate to other rows in the same table?” Choosing the right join depends on whether you need all combinations, only matches, or orphan rows from a specific side.

Ex – 4

SELECT Queries

1. Aim

To create an `employee` table, insert ten sample records, and retrieve data with `SELECT` using `DISTINCT`, `WHERE` (comparison, `OR`, `BETWEEN`, `IN` / `NOT IN`), and `ORDER BY` in ascending and descending order.

2. Theory

2.1 SELECT

`SELECT` reads rows from a relation. `SELECT *` returns every column; naming columns returns only those attributes. A query without a `WHERE` clause returns every row that currently exists in the table.

2.2 DISTINCT

`SELECT DISTINCT` removes duplicate values from the result. `SELECT department_no` lists a department once per employee; `SELECT DISTINCT department_no` lists each department number only once. Use `DISTINCT` when the question is about unique values, not about every row.

2.3 WHERE

The `WHERE` clause keeps rows that satisfy a condition. Comparison operators such as > and <> filter numbers and strings. `OR` keeps a row if either predicate is true (city is Delhi or Mumbai). City values in this lab are stored in title case and matched with the same casing so the filters work in SQLite without extra case-conversion functions.

2.4 BETWEEN

`BETWEEN low AND high` is an inclusive range: the endpoints are part of the result. `NOT BETWEEN` is the complement — salaries strictly below 30000 or strictly above 50000. It is equivalent to `salary < 30000 OR salary > 50000` when the range is 30000 to 50000.

2.5 IN

`IN (…)` tests membership in a list. `city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore')` keeps those four cities. `NOT IN` keeps every city that is not in the list — here Bangalore, Hyderabad, and Pune remain when Mumbai, Delhi, and Chennai are excluded.

2.6 ORDER BY

`ORDER BY` sorts the result. `ASC` (the default) lists names A→Z; `DESC` lists them Z→A. Sorting does not change stored rows; it only changes the order of the result set.

3. Schema

One table, `employee`, holds eight attributes. `sr_no` is the employee number (primary key). `manager` stores the manager's name, or `NULL` for the top manager.

Attribute	Type	Constraint	Role
sr_no	INTEGER	PRIMARY KEY	Employee number
employee_name	TEXT	NOT NULL	Employee name
job	TEXT	NOT NULL	Job title
manager	TEXT	NULL allowed	Manager name
hire_date	DATE	NOT NULL	Date of joining
salary	NUMERIC(10,2)	NOT NULL, CHECK > 0	Monthly salary
department_no	INTEGER	NOT NULL	Department number
city	TEXT	NOT NULL	Work city

4. Procedure

1. Drop `employee` if it already exists, so the script can be re-run.
2. Create the table with the eight attributes listed above.
3. Insert ten employees with mixed salaries, cities, and department numbers.
4. Select all columns from `employee`.
5. Select distinct department numbers, then all department numbers without `DISTINCT`.
6. Filter by salary (> 30000, `BETWEEN`, `NOT BETWEEN`).
7. Filter by city (not Delhi, Delhi or Mumbai, `IN`, `NOT IN`).
8. Sort the full table by employee name descending, then ascending.

5. Source Code

SQLite script used in the lab compiler (`09-09-2026/employee.sql`). For the class SQL Server, run `employee.sqlserver.sql`.

```
-- ============================================================
-- DBMS Lab · 09-09-2026
-- Employee table: CREATE, seed 10 rows, then SELECT queries
-- DISTINCT, WHERE, BETWEEN, IN, ORDER BY
-- SQLite  (sql.js compiler / DB Browser / sqlite3)
-- For the class SQL Server, run employee.sqlserver.sql instead.
-- ============================================================

DROP TABLE IF EXISTS employee;

-- ------------------------------------------------------------
-- 1. Create the employee table
-- ------------------------------------------------------------
CREATE TABLE employee (
    sr_no          INTEGER PRIMARY KEY,
    employee_name  TEXT NOT NULL,
    job            TEXT NOT NULL,
    manager        TEXT,
    hire_date      DATE NOT NULL,
    salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
    department_no  INTEGER NOT NULL,
    city           TEXT NOT NULL
);

-- ------------------------------------------------------------
-- 2. Insert 10 sample rows
--    • salaries below, inside, and above 30000–50000
--    • cities: Delhi, Mumbai, Chennai, Bangalore, Hyderabad, Pune
--    • mixed department numbers (some duplicates)
--    • Rajesh Kumar is the top manager (manager IS NULL)
-- ------------------------------------------------------------
INSERT INTO employee
    (sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
```

```
VALUES
    (1,  'Rajesh Kumar',    'General Manager',    NULL,            '2018-03-12', 85000.00, 10, 'Delhi'),
    (2,  'Ananya Sharma',   'Software Engineer',  'Rajesh Kumar',  '2021-07-01', 45000.00, 20, 'Bangalore'),
    (3,  'Rohan Mehta',     'Accountant',         'Rajesh Kumar',  '2020-01-15', 28000.00, 30, 'Mumbai'),
    (4,  'Priya Nair',      'HR Executive',       'Rajesh Kumar',  '2022-04-20', 32000.00, 40, 'Chennai'),
    (5,  'Vikram Singh',    'Sales Manager',      'Rajesh Kumar',  '2019-11-08', 52000.00, 50, 'Hyderabad'),
    (6,  'Fatima Khan',     'Software Engineer',  'Ananya Sharma', '2023-02-14', 38000.00, 20, 'Bangalore'),
    (7,  'Arjun Patel',     'Clerk',              'Rohan Mehta',   '2024-06-01', 18000.00, 30, 'Pune'),
    (8,  'Neha Gupta',      'Marketing Analyst',  'Rajesh Kumar',  '2021-09-10', 35000.00, 60, 'Delhi'),
    (9,  'Karan Iyer',      'Database Admin',     'Ananya Sharma', '2020-12-05', 48000.00, 20, 'Mumbai'),
    (10, 'Meera Joshi',     'Receptionist',       'Priya Nair',    '2023-08-22', 22000.00, 40, 'Chennai');

-- ------------------------------------------------------------
-- 3. Select all details from employee
-- ------------------------------------------------------------
-- All employee rows
SELECT * FROM employee;

-- ------------------------------------------------------------
-- 4. Select only DISTINCT department numbers
-- ------------------------------------------------------------
-- Distinct department numbers
SELECT DISTINCT department_no FROM employee;

-- ------------------------------------------------------------
-- 5. Select all department numbers (no DISTINCT)
-- ------------------------------------------------------------
-- All department numbers
SELECT department_no FROM employee;

-- ------------------------------------------------------------
-- 6. Employee name and salary where salary > 30000
-- ------------------------------------------------------------
-- Salary greater than 30000
SELECT employee_name, salary
FROM employee
WHERE salary > 30000;

-- ------------------------------------------------------------
-- 7. Employee name and hire date where city is NOT Delhi
-- ------------------------------------------------------------
-- City is not Delhi
SELECT employee_name, hire_date
FROM employee
WHERE city <> 'Delhi';

-- ------------------------------------------------------------
-- 8. Employee name and hire date where city is Delhi OR Mumbai
-- ------------------------------------------------------------
-- City is Delhi or Mumbai
SELECT employee_name, hire_date
FROM employee
WHERE city = 'Delhi' OR city = 'Mumbai';

-- ------------------------------------------------------------
-- 9. Employee number and name where salary BETWEEN 30000 AND 50000
-- ------------------------------------------------------------
-- Salary between 30000 and 50000
SELECT sr_no, employee_name
FROM employee
WHERE salary BETWEEN 30000 AND 50000;

-- ------------------------------------------------------------
-- 10. Employee number and name where salary NOT BETWEEN 30000 AND 50000
-- ------------------------------------------------------------
-- Salary not between 30000 and 50000
SELECT sr_no, employee_name
FROM employee
WHERE salary NOT BETWEEN 30000 AND 50000;

-- ------------------------------------------------------------
-- 11. All details where city IN (Delhi, Mumbai, Chennai, Bangalore)
-- ------------------------------------------------------------
-- City in metro list
SELECT *
FROM employee
WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');

-- ------------------------------------------------------------
-- 12. All details where city NOT IN (Mumbai, Delhi, Chennai)
-- ------------------------------------------------------------
-- City not in Mumbai, Delhi, Chennai
SELECT *
FROM employee
WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');

-- ------------------------------------------------------------
-- 13. All details ordered by employee name DESC
-- ------------------------------------------------------------
-- Employees in descending name order
```

```
SELECT *
FROM employee
ORDER BY employee_name DESC;

-- ------------------------------------------------------------
-- 14. All details ordered by employee name ASC
-- ------------------------------------------------------------
-- Employees in ascending name order
SELECT *
FROM employee
ORDER BY employee_name ASC;
```

6. Output

The script was executed in SQLite in memory. `CREATE` and `INSERT` print a status line; each `SELECT` prints its result set. `NULL` in the manager column is shown as an empty cell.

```
SQL> DROP TABLE IF EXISTS employee;
-- OK

SQL> CREATE TABLE employee (
        sr_no          INTEGER PRIMARY KEY,
        employee_name  TEXT NOT NULL,
        job            TEXT NOT NULL,
        manager        TEXT,
        hire_date      DATE NOT NULL,
        salary         NUMERIC(10, 2) NOT NULL CHECK (salary > 0),
        department_no  INTEGER NOT NULL,
        city           TEXT NOT NULL
    );
-- OK

SQL> INSERT INTO employee
        (sr_no, employee_name, job, manager, hire_date, salary, department_no, city)
    VALUES
        (1,  'Rajesh Kumar',    'General Manager',    NULL,            '2018-03-12', 85000.00,
    10, 'Delhi'),
        (2,  'Ananya Sharma',   'Software Engineer',  'Rajesh Kumar',  '2021-07-01', 45000.00,
    20, 'Bangalore'),
        (3,  'Rohan Mehta',     'Accountant',         'Rajesh Kumar',  '2020-01-15', 28000.00,
    30, 'Mumbai'),
        (4,  'Priya Nair',      'HR Executive',       'Rajesh Kumar',  '2022-04-20', 32000.00,
    40, 'Chennai'),
        (5,  'Vikram Singh',    'Sales Manager',      'Rajesh Kumar',  '2019-11-08', 52000.00,
    50, 'Hyderabad'),
        (6,  'Fatima Khan',     'Software Engineer',  'Ananya Sharma', '2023-02-14', 38000.00,
    20, 'Bangalore'),
        (7,  'Arjun Patel',     'Clerk',              'Rohan Mehta',   '2024-06-01', 18000.00,
    30, 'Pune'),
        (8,  'Neha Gupta',      'Marketing Analyst',  'Rajesh Kumar',  '2021-09-10', 35000.00,
    60, 'Delhi'),
        (9,  'Karan Iyer',      'Database Admin',     'Ananya Sharma', '2020-12-05', 48000.00,
    20, 'Mumbai'),
        (10, 'Meera Joshi',     'Receptionist',       'Priya Nair',    '2023-08-22', 22000.00,
    40, 'Chennai');
-- 10 row(s) inserted

SQL> SELECT * FROM employee;
sr_no  employee_name  job               manager        hire_date   salary  department_no  city
-----  -------------  ----------------  -------------  ----------  ------  -------------  ---------
1      Rajesh Kumar   General Manager                  2018-03-12  85000   10             Delhi
2      Ananya Sharma  Software Engineer  Rajesh Kumar   2021-07-01  45000   20             Bangalore
3      Rohan Mehta    Accountant         Rajesh Kumar   2020-01-15  28000   30             Mumbai
4      Priya Nair     HR Executive       Rajesh Kumar   2022-04-20  32000   40             Chennai
5      Vikram Singh   Sales Manager      Rajesh Kumar   2019-11-08  52000   50             Hyderabad
6      Fatima Khan    Software Engineer  Ananya Sharma  2023-02-14  38000   20             Bangalore
7      Arjun Patel    Clerk              Rohan Mehta    2024-06-01  18000   30             Pune
8      Neha Gupta     Marketing Analyst  Rajesh Kumar   2021-09-10  35000   60             Delhi
9      Karan Iyer     Database Admin     Ananya Sharma  2020-12-05  48000   20             Mumbai
10     Meera Joshi    Receptionist       Priya Nair     2023-08-22  22000   40             Chennai

SQL> SELECT DISTINCT department_no FROM employee;
department_no
-------------
10
20
30
40
50
60

SQL> SELECT department_no FROM employee;
department_no
-------------
10
20
30
40
50
20
30
60
20
40

SQL> SELECT employee_name, salary
```

```
     FROM employee
     WHERE salary > 30000;
employee_name  salary
-------------  ------
Rajesh Kumar   85000
Ananya Sharma  45000
Priya Nair     32000
Vikram Singh   52000
Fatima Khan    38000
Neha Gupta     35000
Karan Iyer     48000

SQL> SELECT employee_name, hire_date
     FROM employee
     WHERE city <> 'Delhi';
employee_name  hire_date
-------------  ----------
Ananya Sharma  2021-07-01
Rohan Mehta    2020-01-15
Priya Nair     2022-04-20
Vikram Singh   2019-11-08
Fatima Khan    2023-02-14
Arjun Patel    2024-06-01
Karan Iyer     2020-12-05
Meera Joshi    2023-08-22

SQL> SELECT employee_name, hire_date
     FROM employee
     WHERE city = 'Delhi' OR city = 'Mumbai';
employee_name  hire_date
-------------  ----------
Rajesh Kumar   2018-03-12
Rohan Mehta    2020-01-15
Neha Gupta     2021-09-10
Karan Iyer     2020-12-05

SQL> SELECT sr_no, employee_name
     FROM employee
     WHERE salary BETWEEN 30000 AND 50000;
sr_no  employee_name
-----  -------------
2      Ananya Sharma
4      Priya Nair
6      Fatima Khan
8      Neha Gupta
9      Karan Iyer

SQL> SELECT sr_no, employee_name
     FROM employee
     WHERE salary NOT BETWEEN 30000 AND 50000;
sr_no  employee_name
-----  -------------
1      Rajesh Kumar
3      Rohan Mehta
5      Vikram Singh
7      Arjun Patel
10     Meera Joshi

SQL> SELECT *
     FROM employee
     WHERE city IN ('Delhi', 'Mumbai', 'Chennai', 'Bangalore');
sr_no  employee_name  job               manager        hire_date   salary  department_no  city
-----  -------------  ----------------  -------------  ----------  ------  -------------  ---------
1      Rajesh Kumar   General Manager                  2018-03-12  85000   10             Delhi
2      Ananya Sharma  Software Engineer  Rajesh Kumar   2021-07-01  45000   20             Bangalore
3      Rohan Mehta    Accountant         Rajesh Kumar   2020-01-15  28000   30             Mumbai
4      Priya Nair     HR Executive       Rajesh Kumar   2022-04-20  32000   40             Chennai
6      Fatima Khan    Software Engineer  Ananya Sharma  2023-02-14  38000   20             Bangalore
8      Neha Gupta     Marketing Analyst  Rajesh Kumar   2021-09-10  35000   60             Delhi
9      Karan Iyer     Database Admin     Ananya Sharma  2020-12-05  48000   20             Mumbai
10     Meera Joshi    Receptionist       Priya Nair     2023-08-22  22000   40             Chennai

SQL> SELECT *
     FROM employee
     WHERE city NOT IN ('Mumbai', 'Delhi', 'Chennai');
sr_no  employee_name  job               manager        hire_date   salary  department_no  city
-----  -------------  ----------------  -------------  ----------  ------  -------------  ---------
2      Ananya Sharma  Software Engineer  Rajesh Kumar   2021-07-01  45000   20             Bangalore
5      Vikram Singh   Sales Manager      Rajesh Kumar   2019-11-08  52000   50             Hyderabad
6      Fatima Khan    Software Engineer  Ananya Sharma  2023-02-14  38000   20             Bangalore
7      Arjun Patel    Clerk              Rohan Mehta    2024-06-01  18000   30             Pune

SQL> SELECT *
     FROM employee
     ORDER BY employee_name DESC;
sr_no  employee_name  job               manager        hire_date   salary  department_no  city
-----  -------------  ----------------  -------------  ----------  ------  -------------  ---------
5      Vikram Singh   Sales Manager      Rajesh Kumar   2019-11-08  52000   50             Hyderabad
3      Rohan Mehta    Accountant         Rajesh Kumar   2020-01-15  28000   30             Mumbai
1      Rajesh Kumar   General Manager                  2018-03-12  85000   10             Delhi
4      Priya Nair     HR Executive       Rajesh Kumar   2022-04-20  32000   40             Chennai
8      Neha Gupta     Marketing Analyst  Rajesh Kumar   2021-09-10  35000   60             Delhi
10     Meera Joshi    Receptionist       Priya Nair     2023-08-22  22000   40             Chennai
9      Karan Iyer     Database Admin     Ananya Sharma  2020-12-05  48000   20             Mumbai
6      Fatima Khan    Software Engineer  Ananya Sharma  2023-02-14  38000   20             Bangalore
7      Arjun Patel    Clerk              Rohan Mehta    2024-06-01  18000   30             Pune
2      Ananya Sharma  Software Engineer  Rajesh Kumar   2021-07-01  45000   20             Bangalore

SQL> SELECT *
     FROM employee
     ORDER BY employee_name ASC;
sr_no  employee_name  job               manager        hire_date   salary  department_no  city
```

```
-----  -------------  -----------------  -------------  ----------  ------  -------------  ---------
2      Ananya Sharma  Software Engineer  Rajesh Kumar   2021-07-01  45000   20             Bangalore
7      Arjun Patel    Clerk              Rohan Mehta    2024-06-01  18000   30             Pune
6      Fatima Khan    Software Engineer  Ananya Sharma  2023-02-14  38000   20             Bangalore
9      Karan Iyer     Database Admin     Ananya Sharma  2020-12-05  48000   20             Mumbai
10     Meera Joshi    Receptionist       Priya Nair     2023-08-22  22000   40             Chennai
8      Neha Gupta     Marketing Analyst  Rajesh Kumar   2021-09-10  35000   60             Delhi
4      Priya Nair     HR Executive       Rajesh Kumar   2022-04-20  32000   40             Chennai
1      Rajesh Kumar   General Manager                   2018-03-12  85000   10             Delhi
3      Rohan Mehta    Accountant         Rajesh Kumar   2020-01-15  28000   30             Mumbai
5      Vikram Singh   Sales Manager      Rajesh Kumar   2019-11-08  52000   50             Hyderabad
```

7. Results

Ten employees load successfully. The filters return non-empty, distinct subsets as expected for this seed data:

Query	Rows	What the result shows
SELECT *	10	Every employee and every column
DISTINCT department_no	6	Departments 10, 20, 30, 40, 50, 60
department_no (no DISTINCT)	10	Duplicates kept (dept 20 appears three times)
salary > 30000	7	Excludes 18000, 22000, 28000
city <> Delhi	8	Keeps everyone except Rajesh and Neha
city Delhi OR Mumbai	4	Rajesh, Rohan, Neha, Karan
salary BETWEEN 30000 AND 50000	5	Inclusive range; excludes low and high pay
salary NOT BETWEEN 30000 AND 50000	5	Complement of the BETWEEN query
city IN (four metros)	8	Drops Hyderabad and Pune
city NOT IN (Mumbai, Delhi, Chennai)	4	Bangalore, Hyderabad, Pune remain
ORDER BY name DESC / ASC	10	Same ten rows; opposite sort orders

`DISTINCT` collapses ten department values to six unique numbers. `BETWEEN` and `NOT BETWEEN` partition the table with no overlap (5 + 5 = 10). City membership tests leave Hyderabad and Pune only in the `NOT IN` result, which is the check that the seed data includes cities outside the four-metro list.

8. Conclusion

A single `SELECT` can project columns, remove duplicates, filter with comparison and set predicates, and sort the result. `DISTINCT` answers “which unique values exist?” `WHERE` answers “which rows match this condition?” `BETWEEN` and `IN` express ranges and lists clearly. `ORDER BY` changes presentation only. Together these clauses are the core of everyday retrieval in SQL.